

Министерство науки и высшего образования Российской Федерации

Федеральное государственное автономное образовательное учреждение
высшего образования
«СЕВЕРО-КАВКАЗСКИЙ ФЕДЕРАЛЬНЫЙ УНИВЕРСИТЕТ»

Институт перспективной инженерии
Департамента цифровых, робототехнических систем и электроники

ОТЧЕТ
ПО ЛАБОРАТОРНОЙ РАБОТЕ №2
дисциплины «Искусственный интеллект и машинное обучение»
Вариант 12

Выполнила:
Коробка В.А.,
2 курс, группа ИВТ-б-о-24-1,
09.03.01 «Информатика и
вычислительная техника»,
направленность (профиль)
«Программное обеспечение средств
вычислительной техники и
автоматизированных систем», очная
форма обучения

(подпись)

Руководитель практики:
Воронкин Роман Александрович,
доцент департамента цифровых,
робототехнических систем и
электроники института перспективной
инженерии

(подпись)

Отчет защищен с оценкой _____ Дата защиты _____
Ставрополь, 2026 г.

Тема: Основы работы с библиотекой NumPy

Цель: исследовать базовые возможности библиотеки NumPy языка программирования Python.

Ссылка на github: <https://github.com/13r4lera/ML2.git>

Порядок выполнения работы

Были проработаны все примеры лабораторной работы.

Было выполнено задание 1 лабораторной работы. Был создан массив NumPy размером 3×3 , содержащий числа от 1 до 9. Все элементы массива были умножены на 2 с использованием поэлементных операций NumPy. Затем с помощью логической индексации элементы, значения которых превышали 10, были заменены на 0. В результате получен преобразованный массив с обновлёнными значениями.

Условие задания 1

Создайте массив NumPy размером 3×3 , содержащий числа от 1 до 9. Умножьте все элементы массива на 2, а затем замените все элементы больше 10 на 0. Выведите итоговый массив.

[2]:

```
import numpy as np
arr = np.array([
    [1, 2, 3],
    [4, 5, 6],
    [7, 8, 9]
])
arr = arr * 2
arr[arr > 10] = 0
print(arr)
```

```
[[ 2  4  6]
 [ 8 10  0]
 [ 0  0  0]]
```

Рисунок 1. Выполнение задания 1

Было выполнено задание 2. В ходе выполнения задания был создан массив NumPy из 20 случайных целых чисел в диапазоне от 1 до 100. С помощью логической индексации были найдены элементы, делящиеся на 5 без остатка.

Найденные значения были заменены на -1 , после чего выведен обновлённый массив с изменёнными элементами.

Условие задания 2

Создайте массив NumPy из 20 случайных целых чисел от 1 до 100. Найдите и выведите все элементы, которые делятся на 5 без остатка. Затем замените их на -1 и выведите обновленный массив.

[3]:

```
arr = np.random.randint(1, 101, 20)
print("Исходный массив:")
print(arr)
print("\nЧисла, делящиеся на 5:")
print(arr[arr % 5 == 0])
arr[arr % 5 == 0] = -1
print("\nОбновленный массив:")
print(arr)
```

Исходный массив:

```
[100  50  90  50   1  31  43   7  72  82  57   7  55  98  95   9  68  85
 24   9]
```

Числа, делящиеся на 5:

```
[100  50  90  50  55  95  85]
```

Обновленный массив:

```
[-1 -1 -1 -1   1  31  43   7  72  82  57   7 -1  98 -1   9  68 -1  24   9]
```

Рисунок 2. Выполнение задания 2

Было выполнено задание 3. В ходе выполнения задания были созданы два одномерных массива NumPy размером 1×5 , заполненные случайными числами от 0 до 50. Эти массивы были объединены в один двумерный массив размером 2×5 с помощью вертикального объединения. Затем полученный массив был разделён обратно на два исходных массива. В результате выполненных операций были получены и выведены все промежуточные и итоговые данные.

Условие задания 3

Создайте два массива NumPy размером 1x5, заполненные случайными числами от 0 до 50.

Объедините эти массивы в один двумерный массив (по строкам). Разделите полученный массив на два массива, каждый из которых содержит 5 элементов.

Выведите все промежуточные и итоговые результаты.

[4]:

```
a = np.random.randint(0, 51, 5)
b = np.random.randint(0, 51, 5)
print("Массив A:", a)
print("Массив B:", b)
c = np.vstack((a, b))
print("\nОбъединенный массив:")
print(c)
a_new, b_new = np.vsplit(c, 2)
print("\nРазделенный массив A:")
print(a_new.flatten())
print("Разделенный массив B:")
print(b_new.flatten())
```

Массив A: [35 28 41 46 44]

Массив B: [47 30 3 27 45]

Объединенный массив:

[[35 28 41 46 44]

[47 30 3 27 45]]

Разделенный массив A:

[35 28 41 46 44]

Разделенный массив B:

[47 30 3 27 45]

Рисунок 3. Выполнение задания 3

Было выполнено задание 4. В ходе выполнения задания был создан массив NumPy из 50 чисел, равномерно распределённых на интервале от -10 до 10 с помощью функции `linspace`. Далее были вычислены три характеристики: сумма всех элементов массива, сумма положительных элементов и сумма отрицательных элементов. Для выделения положительных и отрицательных значений использовалась логическая индексация. В результате получены соответствующие числовые значения для каждого случая.

Условие задания 4

Создайте массив из 50 чисел, равномерно распределенных от -10 до 10. Вычислите сумму всех элементов, сумму положительных элементов и сумму отрицательных элементов. Выведите результаты.

[6]:

```
arr = np.linspace(-10, 10, 50)
print("Массив:")
print(arr)
sum_all = np.sum(arr)
sum_pos = np.sum(arr[arr > 0])
sum_neg = np.sum(arr[arr < 0])
print("\nСумма всех элементов:", sum_all)
print("Сумма положительных элементов:", sum_pos)
print("Сумма отрицательных элементов:", sum_neg)
```

Массив:

```
[ -10.          -9.59183673 -9.18367347 -8.7755102  -8.36734694
  -7.95918367 -7.55102041 -7.14285714 -6.73469388 -6.32653061
  -5.91836735 -5.51020408 -5.10204082 -4.69387755 -4.28571429
  -3.87755102 -3.46938776 -3.06122449 -2.65306122 -2.24489796
  -1.83673469 -1.42857143 -1.02040816 -0.6122449  -0.20408163
   0.20408163  0.6122449   1.02040816  1.42857143  1.83673469
   2.24489796  2.65306122  3.06122449  3.46938776  3.87755102
   4.28571429  4.69387755  5.10204082  5.51020408  5.91836735
   6.32653061  6.73469388  7.14285714  7.55102041  7.95918367
   8.36734694  8.7755102   9.18367347  9.59183673 10.         ]
```

Сумма всех элементов: 7.105427357601002e-15

Сумма положительных элементов: 127.55102040816328

Сумма отрицательных элементов: -127.55102040816327

Рисунок 4. Выполнение задания 4

Было выполнено задание 5. В ходе выполнения задания были созданы две матрицы NumPy: единичная матрица размером 4×4 и диагональная матрица с заданными элементами [5, 10, 15, 20]. Сумма элементов единичной матрицы была рассчитана и составила 4, так как на главной диагонали находятся только единицы. Сумма элементов диагональной матрицы составила 50, поскольку все ненулевые значения расположены на главной диагонали.

Условие задания 5

Создайте:

- Единичную матрицу размером 4x4.
- Диагональную матрицу размером 4x4 с диагональными элементами [5, 10, 15, 20] (не использовать циклы).

Найдите сумму всех элементов каждой из этих матриц и сравните результаты.

[7]:

```
I = np.eye(4)
D = np.diag([5, 10, 15, 20])
sum_I = I.sum()
sum_D = D.sum()
print(I)
print(D)
print("Сумма элементов единичной матрицы:", sum_I)
print("Сумма элементов диагональной матрицы:", sum_D)
```

```
[[1.  0.  0.  0.]
 [0.  1.  0.  0.]
 [0.  0.  1.  0.]
 [0.  0.  0.  1.]]
[[ 5  0  0  0]
 [ 0 10  0  0]
 [ 0  0 15  0]
 [ 0  0  0 20]]
```

Сумма элементов единичной матрицы: 4.0

Сумма элементов диагональной матрицы: 50

Рисунок 5. Выполнение задания 5

Было выполнено задание 6. В ходе выполнения задания были созданы две квадратные матрицы NumPy размером 3×3 , заполненные случайными целыми числами от 1 до 20. С использованием поэлементных операций были вычислены их сумма, разность и произведение. Для сложения и вычитания применялись стандартные арифметические операции NumPy, а для умножения поэлементное умножение массивов.

Создайте две квадратные матрицы NumPy размером 3x3, заполненные случайными целыми числами от 1 до 20. Вычислите и выведите: их сумму, их разность, их поэлементное произведение.

[8]:

```
A = np.random.randint(1, 21, (3, 3))
B = np.random.randint(1, 21, (3, 3))

print("A:\n", A)
print("\nB:\n", B)

print("\nСумма:\n", A + B)
print("\nРазность:\n", A - B)
print("\nПоэлементное произведение:\n", A * B)
```

A:

```
[[18  6  2]
 [17  4  1]
 [15 19 11]]
```

B:

```
[[ 2 11  6]
 [ 7  7  5]
 [ 3  1 14]]
```

Сумма:

```
[[20 17  8]
 [24 11  6]
 [18 20 25]]
```

Разность:

```
[[16 -5 -4]
 [10 -3 -4]
 [12 18 -3]]
```

Поэлементное произведение:

```
[[ 36 66 12]
 [119 28  5]
 [ 45 19 154]]
```

Рисунок 6. Выполнение задания 6

Было выполнено задание 7. В ходе выполнения задания были созданы две матрицы NumPy: первая размером 2×3 , вторая размером 3×2 , заполненные случайными числами от 1 до 10. Затем было выполнено матричное умножение с использованием оператора '@' и функции np.dot, которые дают одинаковый результат. В результате получена новая матрица размером 2×2 , содержащая произведение исходных матриц.

Условие задания 7

Создайте две матрицы NumPy:

- Первую размером 2x3, заполненную случайными числами от 1 до 10.
- Вторую размером 3x2, заполненную случайными числами от 1 до 10.

Выполните матричное умножение (@ или np.dot) и выведите результат.

[9]:

```
A = np.random.randint(1, 11, (2, 3))
B = np.random.randint(1, 11, (3, 2))

print("Матрица A:\n", A)
print("\nМатрица B:\n", B)

C = A @ B
print("\nРезультат умножения:\n", C)
C = np.dot(A, B)
print("\nРезультат умножения:\n", C)
```

Матрица A:

```
[[1 3 7]
 [6 3 3]]
```

Матрица B:

```
[[9 2]
 [9 3]
 [8 1]]
```

Результат умножения:

```
[[ 92 18]
 [105 24]]
```

Результат умножения:

```
[[ 92 18]
 [105 24]]
```

Рисунок 7. Выполнение задания 7

Было выполнено задание 8. В ходе выполнения задания была создана случайная квадратная матрица 3×3 с элементами от 1 до 10. Затем с помощью функции `np.linalg.det` был вычислен её определитель, который оказался отличным от нуля, что означает существование обратной матрицы. После этого с использованием `np.linalg.inv` была найдена обратная матрица.

Условие задания 8

Создайте случайную квадратную матрицу 3x3. Найдите и выведите:

- Определитель этой матрицы
- Обратную матрицу (если существует, иначе выведите сообщение, что матрица вырождена)

Используйте функции `np.linalg.det` и `np.linalg.inv`.

[10]:

```
A = np.random.randint(1, 11, (3, 3))
print("Матрица A:\n", A)
det = np.linalg.det(A)
print("\nОпределитель:", det)

if det != 0:
    inv_A = np.linalg.inv(A)
    print("\nОбратная матрица:\n", inv_A)
else:
    print("\nМатрица вырождена (обратной матрицы не существует)")
```

Матрица A:

```
[[ 6  7  3]
 [ 3  1 10]
 [ 5  9  4]]
```

Определитель: -184.00000000000006

Обратная матрица:

```
[[ 0.4673913  0.00543478 -0.36413043]
 [-0.20652174 -0.04891304  0.27717391]
 [-0.11956522  0.10326087  0.08152174]]
```

Рисунок 8. Выполнение задания 8

Было выполнено задание 9. В ходе выполнения задания была создана квадратная матрица NumPy размером 4×4, заполненная случайными целыми числами от 1 до 50. Затем была получена транспонированная матрица с помощью операции транспонирования. После этого был вычислен след матрицы с использованием функции `np.trace`, который представляет собой сумму элементов главной диагонали.

Условие задания 9

Создайте матрицу NumPy размером 4x4, содержащую случайные целые числа от 1 до 50. Выведите:

- Исходную матрицу
- Транспонированную матрицу
- След матрицы (сумму элементов на главной диагонали)

Используйте `np.trace` для нахождения следа.

[11]:

```
A = np.random.randint(1, 51, (4, 4))
print("Исходная матрица:\n", A)
print("\nТранспонированная матрица:\n", A.T)
trace = np.trace(A)
print("\nСлед матрицы:", trace)
```

Исходная матрица:

```
[[36 16 50 45]
 [ 4 36 36 15]
 [26 40 24 43]
 [38  9 44 39]]
```

Транспонированная матрица:

```
[[36  4 26 38]
 [16 36 40  9]
 [50 36 24 44]
 [45 15 43 39]]
```

След матрицы: 135

Рисунок 9. Выполнение задания 9

Было выполнено задание 10. В ходе выполнения задания система линейных уравнений была записана в матричном виде ($A * X = B$), где матрица (A) содержит коэффициенты при неизвестных, а вектор (B) правые части уравнений. Для нахождения решения использовалась функция `np.linalg.solve`, позволяющая вычислить значения неизвестных без ручных преобразований.

Условие задания 10

Решите систему линейных уравнений вида:

$$\begin{cases} 2x + 3y - z = 5 \\ 4x - y + 2z = 6 \\ -3x + 5y + 4z = -2 \end{cases}$$

Используйте матричное представление $Ax = B$, где A - матрица коэффициентов, x - вектор неизвестных, B - вектор правой части. Решите систему с помощью `np.linalg.solve` и выведите результат.

```
A = np.array([
    [2, 3, -1],
    [4, -1, 2],
    [-3, 5, 4]
])
B = np.array([5, 6, -2])
X = np.linalg.solve(A, B)
print("Решение системы:")
print("x =", X[0])
print("y =", X[1])
print("z =", X[2])
```

```
Решение системы:
x = 1.6396396396396398
y = 0.5765765765765767
z = 0.009009009009008993
```

Рисунок 10. Выполнение задание 10

Было выполнено задание 11. В ходе выполнения задания задача о продаже билетов была сведена к системе линейных уравнений, учитывающей общее количество билетов, разницу между числом стандартных и премиум-билетов, а также общий доход с учётом различной стоимости категорий. Система была записана в матричном виде ($A * X = B$) и решена с использованием функции `np.linalg.solve`. В результате установлено, что было продано 300 стандартных билетов, 200 премиум-билетов и VIP-билеты не продавались.

Условие задачи 11 по варианту 12

Продажа билетов. На концерт продано 500 билетов трех категорий: стандартные, премиум и VIP. Билеты премиум стоят в 1.5 раза дороже стандартных, а VIP - в 2 раза дороже премиум. Общий доход от продажи составил 600 000 рублей. Сколько билетов каждого типа было продано, если известно, что стандартных билетов было на 100 больше, чем премиум?

```
A = np.array([
    [1, 1, 1],
    [1, -1, 0],
    [1, 1.5, 3]
])
B = np.array([500, 100, 600])
X = np.linalg.solve(A, B)

print("Стандартные:", int(X[0]))
print("Премиум:", int(X[1]))
print("VIP:", int(X[2]))
```

```
Стандартные: 300
Премиум: 200
VIP: 0
```

Рисунок 11. Выполнение задания 11

Ответы на контрольные вопросы

1. Каково назначение библиотеки NumPy?

Библиотека NumPy предназначена для эффективной работы с многомерными массивами данных и выполнения быстрых математических, статистических и линейно-алгебраических вычислений в языке Python.

2. Что такое массивы ndarray?

Массив ndarray - это основной тип данных библиотеки NumPy, представляющий собой многомерный массив элементов одного типа, размещённых в памяти компактно и упорядоченно.

3. Как осуществляется доступ к частям многомерного массива?

Доступ к частям многомерного массива выполняется с помощью индексации и срезов, аналогично спискам Python, но с указанием индексов по каждой оси массива. Для двумерного массива можно обратиться к элементу как `a[i, j]`, к строке `a[i, :]`, к столбцу `a[:, j]`, а к подмассиву `a[1:3, 0:2]`.

4. Как осуществляется расчет статистик по данным?

Наиболее часто используются функции `np.sum()`, `np.mean()`, `np.min()`, `np.max()`, `np.std()`, `np.var()` и `np.median()`, позволяющие вычислять сумму, среднее значение, минимум, максимум, стандартное отклонение, дисперсию и медиану.

5. Как выполняется выборка данных из массивов ndarray?

Выборка данных из массивов ndarray выполняется с помощью обычной индексации, срезов, логических масок и продвинутой индексации. Простая выборка позволяет получать отдельные элементы и диапазоны значений.

6. Приведите основные виды матриц и векторов. Опишите способы их создания в языке Python.

Основными видами матриц являются строка, столбец, квадратная матрица, прямоугольная матрица, единичная, нулевая, диагональная, треугольная и симметричная матрицы, а основными видами векторов — вектор-строка и вектор-столбец. В Python с использованием NumPy они создаются с помощью

функции `np.array()` из вложенных списков, а также специальных функций: `np.zeros()` для нулевых матриц, `np.ones()` для матриц из единиц, `np.eye()` для единичной матрицы, `np.diag()` для диагональной матрицы, `np.arange()` и `np.linspace()` для векторов с заданным диапазоном значений. При необходимости форма массива изменяется с помощью метода `reshape()`

7. Как выполняется транспонирование матриц?

Транспонирование матрицы выполняется путём замены её строк на столбцы и столбцов на строки, то есть элемент с индексом (i, j) переходит на позицию (j, i) . В результате, если исходная матрица имела размер $m \times n$, то после транспонирования она будет иметь размер $n \times m$.

8. Приведите свойства операции транспонирования матриц.

Транспонирование суммы матриц равно сумме транспонированных матриц, то есть $(A + B)^T = A^T + B^T$; транспонирование произведения матриц выполняется в обратном порядке, то есть $(AB)^T = B^T A^T$; двойное транспонирование возвращает исходную матрицу, то есть $(A^T)^T = A$; транспонирование произведения матрицы на число не изменяет коэффициент, то есть $(kA)^T = kA^T$.

9. Какие имеются средства в библиотеке NumPy для выполнения транспонирования матриц?

В библиотеке NumPy для транспонирования матриц используются атрибут `.T` и функция `np.transpose()`.

10. Какие существуют основные действия над матрицами?

К основным действиям над матрицами относятся сложение, вычитание, умножение матрицы на число, поэлементное умножение, матричное умножение, транспонирование, нахождение определителя, обратной матрицы, ранга, следа и решение систем линейных уравнений.

11. Как осуществляется умножение матрицы на число?

Умножение матрицы на число осуществляется путём умножения каждого её элемента на этот коэффициент. Если матрица обозначается как A , а число как k , то результат kA или $A * k$ представляет собой новую матрицу того же размера, в которой каждый элемент увеличен или уменьшен в k раз.

12. Какие свойства операции умножения матрицы на число?

Умножение матрицы на число обладает следующими свойствами: распределительность относительно сложения матриц, то есть $k(A + B) = kA + kB$; распределительность относительно сложения чисел, то есть $(k + m)A = kA + mA$; сочетательность, то есть $k(mA) = (km)A$; а также свойство умножения на единицу и ноль: $1A = A$, $0A = 0$.

13. Как осуществляется операции сложения и вычитания матриц?

Сложение и вычитание матриц выполняются поэлементно, то есть элементы с одинаковыми индексами складываются или вычитаются друг из друга. Такие операции возможны только для матриц одинакового размера, поскольку каждой позиции одной матрицы должна соответствовать такая же позиция другой. В NumPy сложение и вычитание выполняются с помощью стандартных операторов $+$ и $-$.

14. Каковы свойства операций сложения и вычитания матриц?

Сложение матриц обладает свойствами коммутативности $A + B = B + A$, ассоциативности $(A + B) + C = A + (B + C)$, а также наличием нулевой матрицы и противоположной матрицы: $A + 0 = A$, $A + (-A) = 0$. Вычитание матриц не является коммутативным и по сути определяется как сложение первой матрицы с матрицей, противоположной второй: $A - B = A + (-B)$.

15. Какие имеются средства в библиотеке NumPy для выполнения операций сложения и вычитания матриц?

В библиотеке NumPy для выполнения сложения и вычитания матриц используются стандартные арифметические операторы $+$ и $-$, которые выполняют поэлементные действия над массивами одинаковой формы. Кроме того, можно

применять функции `np.add(A, B)` и `np.subtract(A, B)`, которые дают те же результаты, но в функциональной форме.

16. Как осуществляется операция умножения матриц?

Операция умножения матриц выполняется по правилу линейной алгебры: каждый элемент результирующей матрицы вычисляется как сумма произведений элементов строки первой матрицы на соответствующие элементы столбца второй матрицы. Умножение возможно только в том случае, если число столбцов первой матрицы равно числу строк второй. Если матрица A имеет размер $m \times n$, а матрица B $n \times k$, то результат AB будет иметь размер $m \times k$.

17. Каковы свойства операции умножения матриц?

Умножение матриц обладает свойством ассоциативности $(AB)C = A(BC)$ и распределительности относительно сложения $A(B + C) = AB + AC$, $(A + B)C = AC + BC$, однако в общем случае оно не является коммутативным, то есть обычно $AB \neq BA$. Также для единичной матрицы выполняется свойство $AE = EA = A$, а при умножении на нулевую матрицу результатом является нулевая матрица.

18. Какие имеются средства в библиотеке NumPy для выполнения операции умножения матриц?

В NumPy матричное умножение выполняется с помощью оператора `@`, функции `np.dot()` и функции `np.matmul()`. Для двумерных массивов эти способы дают одинаковый результат, например $C = A @ B$ или $C = np.dot(A, B)$. Следует учитывать, что оператор `*` в NumPy означает не матричное, а поэлементное умножение, поэтому для классического произведения матриц необходимо использовать именно `@`, `np.dot()` или `np.matmul()`.

19. Что такое определитель матрицы? Каковы свойства определителя матрицы?

Определитель матрицы — это числовая характеристика квадратной матрицы, которая показывает, является ли матрица вырожденной, и используется при решении систем линейных уравнений, нахождении обратной матрицы и

исследовании линейных преобразований. Если определитель равен нулю, то матрица необратима; если не равен нулю, то обратная матрица существует. К основным свойствам определителя относятся: $\det(AB) = \det(A)\det(B)$, $\det(A^T) = \det(A)$, определитель единичной матрицы равен 1, а при перестановке двух строк определитель меняет знак.

20. Какие имеются средства в библиотеке NumPy для нахождения значения определителя матрицы?

Для нахождения определителя матрицы в библиотеке NumPy используется функция `np.linalg.det()`, которая принимает квадратную матрицу в качестве аргумента и возвращает её определитель. Например, запись `np.linalg.det(A)` позволяет вычислить числовое значение определителя матрицы A .

21. Что такое обратная матрица? Какой алгоритм нахождения обратной матрицы?

Обратная матрица - это такая матрица A^{-1} , которая при умножении на исходную квадратную матрицу A даёт единичную матрицу: $AA^{-1} = A^{-1}A = E$. Обратная матрица существует только для невырожденных матриц, то есть когда их определитель не равен нулю. Один из алгоритмов нахождения обратной матрицы состоит в вычислении определителя, проверке условия $\det(A) \neq 0$, затем нахождении матрицы алгебраических дополнений, её транспонировании и делении на определитель, хотя на практике чаще используются численные методы, реализованные в библиотечных функциях.

22. Каковы свойства обратной матрицы?

Обратная матрица обладает следующими свойствами: обратная к произведению матриц равна произведению обратных матриц в обратном порядке, то есть $(AB)^{-1} = B^{-1}A^{-1}$; обратная к транспонированной матрице равна транспонированной обратной, то есть $(A^T)^{-1} = (A^{-1})^T$; обратная к обратной матрице совпадает с исходной, то есть $(A^{-1})^{-1} = A$; обратная к единичной матрице равна самой единичной матрице.

23. Какие имеются средства в библиотеке NumPy для нахождения обратной матрицы?

Для нахождения обратной матрицы в библиотеке NumPy используется функция `np.linalg.inv()`, которая принимает квадратную невырожденную матрицу и возвращает её обратную.

24. Самостоятельно изучите метод Крамера для решения систем линейных уравнений. Приведите алгоритм решения системы линейных уравнений методом Крамера средствами библиотеки NumPy.

Метод Крамера применяется для решения квадратной системы линейных уравнений $AX = B$, если определитель матрицы коэффициентов A не равен нулю. Алгоритм состоит в следующем: сначала вычисляется $\det(A)$ с помощью `np.linalg.det(A)`; если он равен нулю, метод неприменим; затем для каждого неизвестного формируется матрица A_i , в которой i -й столбец заменён столбцом свободных членов B ; после этого вычисляются определители $\det(A_i)$, а значения неизвестных находятся по формулам $x_i = \det(A_i) / \det(A)$. В NumPy такая реализация выполняется через копирование матрицы, замену столбца и последовательный вызов функции `np.linalg.det()`.

25. Самостоятельно изучите матричный метод для решения систем линейных уравнений. Приведите алгоритм решения системы линейных уравнений матричным методом средствами библиотеки NumPy.

Матричный метод решения системы линейных уравнений основан на записи системы в виде $AX = B$, где A - матрица коэффициентов, X - вектор неизвестных, B - вектор свободных членов. Если матрица A обратима, то решение находится по формуле $X = A^{-1}B$. Алгоритм в NumPy включает создание матрицы A и вектора B , проверку обратимости по определителю `np.linalg.det(A)`, нахождение обратной матрицы с помощью `np.linalg.inv(A)` и последующее матричное умножение $X = A_inv @ B$; однако на практике более устойчивым и предпочтительным способом является использование функции `np.linalg.solve(A,`

В), которая сразу находит решение системы без явного вычисления обратной матрицы.

Вывод: В ходе выполнения лабораторной работы были изучены базовые возможности библиотеки NumPy языка программирования Python для работы с массивами и матрицами. Были освоены методы создания одномерных и многомерных массивов, выполнения поэлементных операций, логической индексации и выборки данных.